

C Programming

16_threads

Потоки (threads)



thread - поток - нить

МНОГОПОТОЧНОСТЬ

Многопотóчность (англ. Multithreading) — свойство платформы (например, операционной системы, виртуальной машины и т. д.) или приложения, состоящее в том, что процесс, порождённый в операционной системе, может состоять из нескольких потоков, выполняющихся «параллельно», то есть без предписанного порядка во времени. При выполнении некоторых задач такое разделение может достичь более эффективного использования ресурсов вычислительной машины.

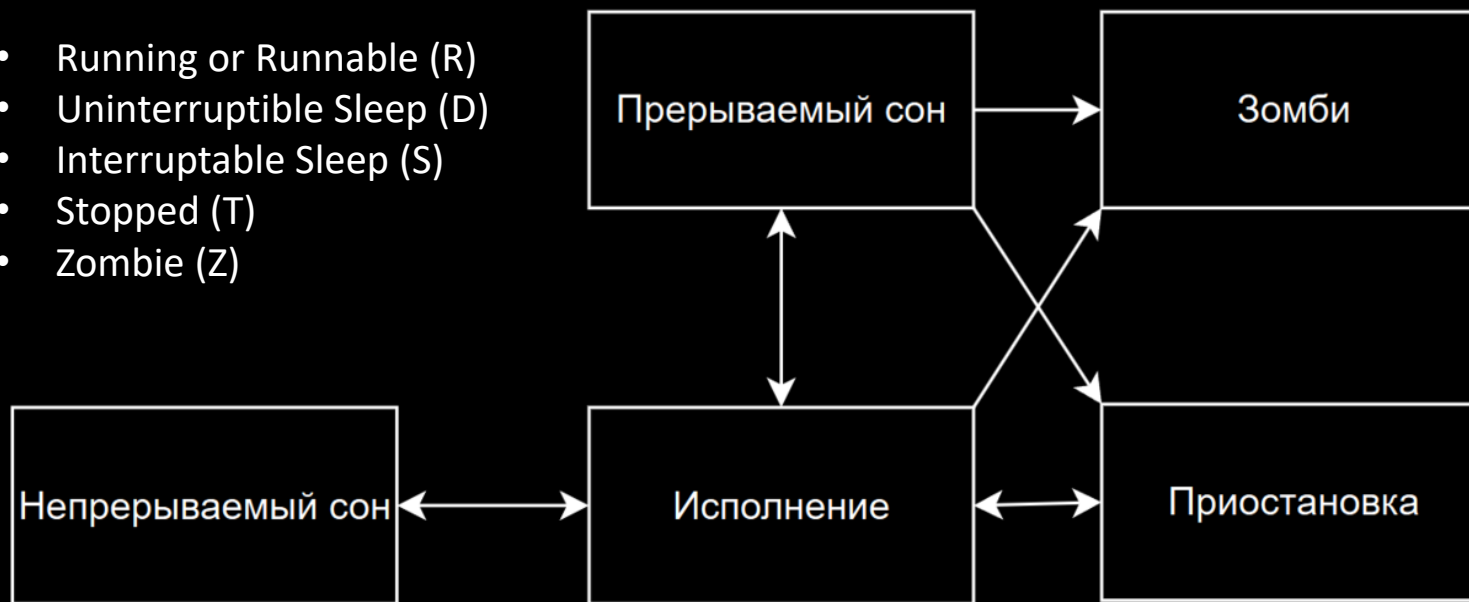
Многопоточность позволяет программе выполнять несколько задач одновременно. Например, когда вы используете веб-браузер: Один поток загружает содержимое веб-страницы (текст, изображения, видео), в то время как другой поток реагирует на ваши действия, такие как движение мыши или прокрутка страницы. Такая многозадачность делает браузер плавным и отзывчивым. Если потоки не используются, браузер может зависать и ждать завершения одной задачи, прежде чем приступить к следующей. Потоки обеспечивают плавность, скорость и отзывчивость работы программы.

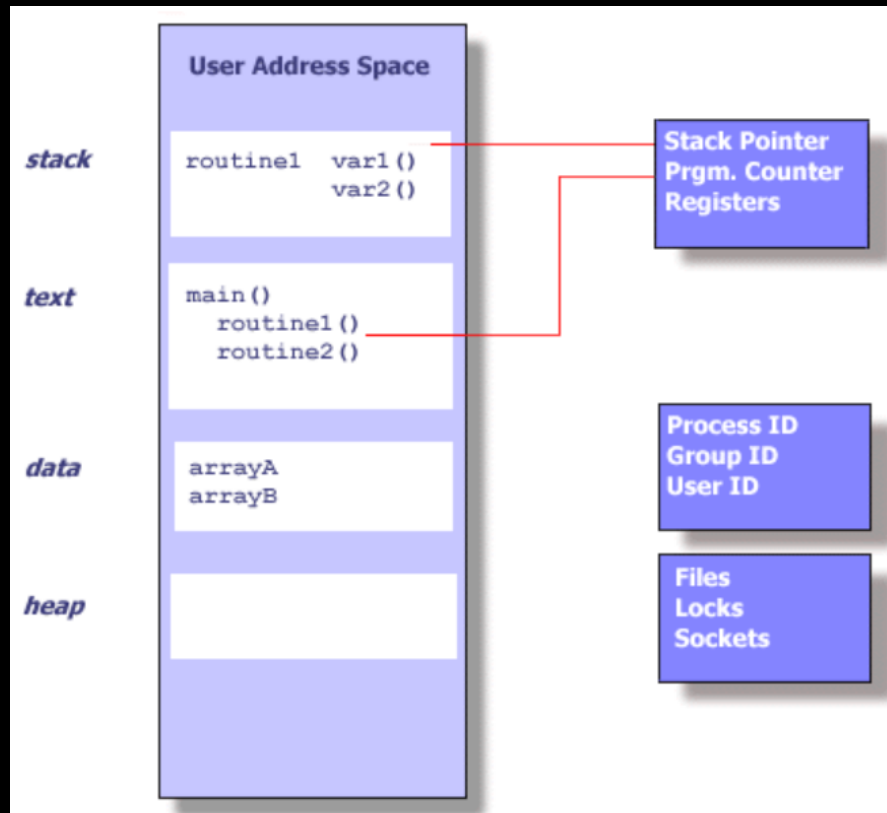
Процессы и потоки

Процессы — это программы, которые выполняются на CPU. Процесс требует больше времени для завершения и изолирован от других процессов, что означает, что он не разделяет с ними память

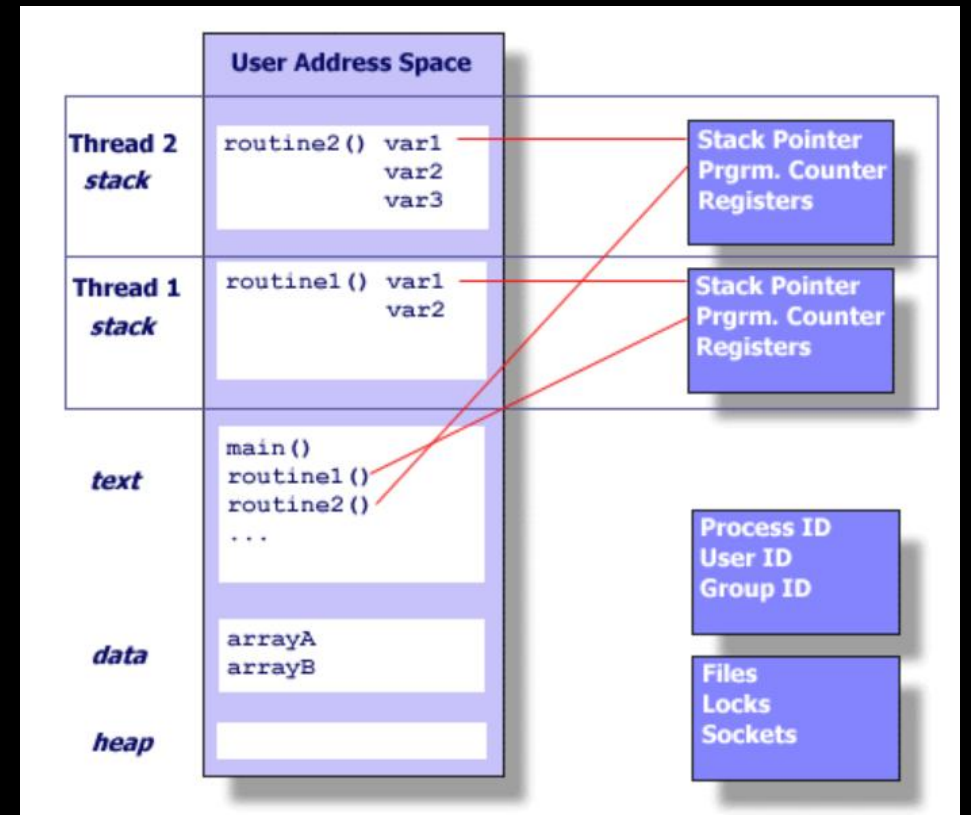
Потоки часто называют «легковесными процессами» (lightweight process), потому что они разделяют некоторые черты процессов, но меньше и быстрее. Каждый поток всегда является частью одного определенного процесса. Поток может находиться в следующих состояниях: Выполняется, Готов, Заблокирован, Сон.

- Running or Runnable (R)
- Uninterruptible Sleep (D)
- Interruptable Sleep (S)
- Stopped (T)
- Zombie (Z)





- Identifying information (e.g. process ID and user ID)
- Environment variables
- Working directory
- Program instructions
- CPU state (e.g. registers, fault info)
- Memory allocation (stack and heap)
- File descriptors
- Signal actions
- Shared libraries
- Inter-process communication related information (such as message queues, pipes, semaphores, or shared memory).



- Stack pointer
- Registers
- Scheduling properties (such as policy or priority)
- Set of pending and blocked signals
- Thread-specific data

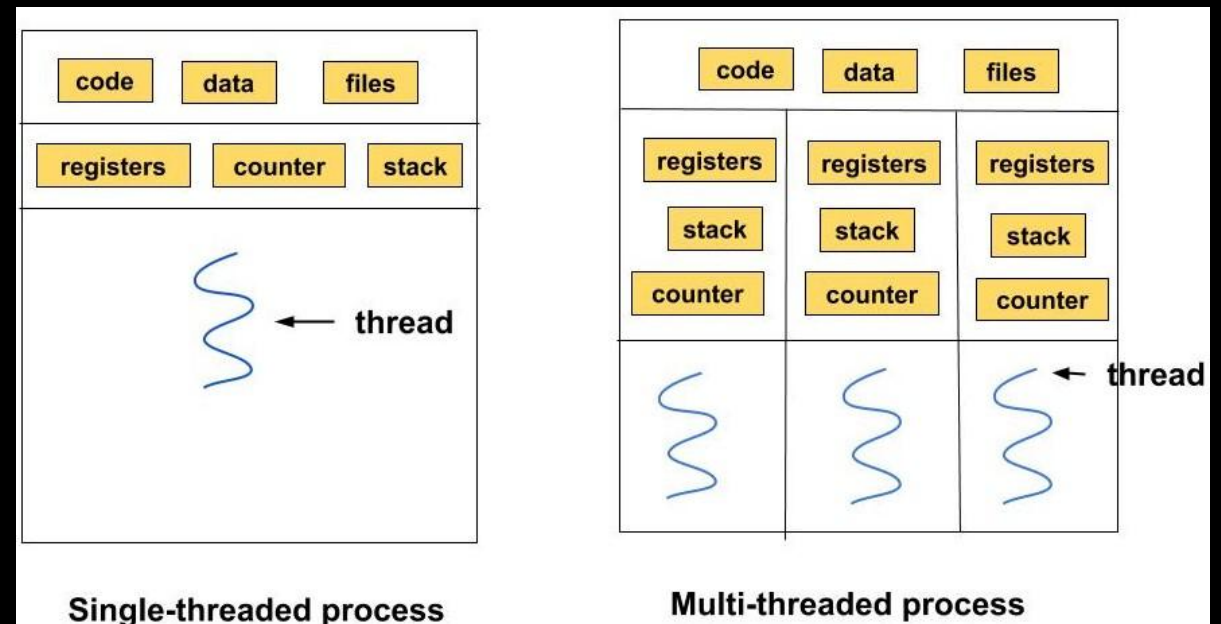
Итого

Процесс — это выполняемая программа, тогда как **поток**:

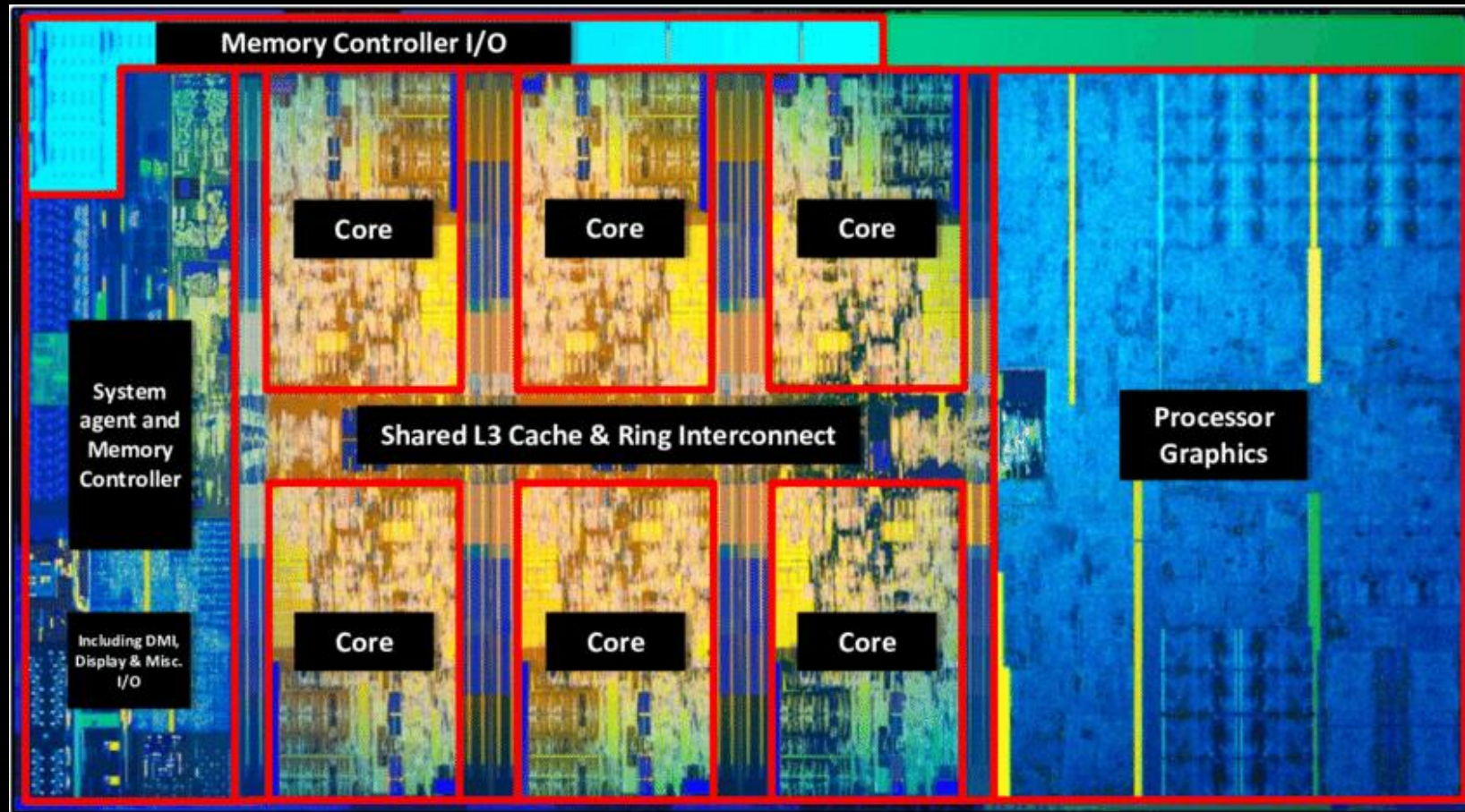
- Существует внутри процесса и использует ресурсы процесса
- Работает пока жив родительский поток
- Дублирует только основные ресурсы, которые ему нужны для независимого планирования, что делает его легким
- Может совместно использовать ресурсы процесса с другими потоками

Поскольку потоки в одном процессе совместно используют ресурсы:

- Изменения, внесенные одним потоком в общие системные ресурсы (например, закрытие файла), будут видны всем другим потокам
- Два указателя с одинаковым значением указывают на одни и те же данные
- Возможно чтение и запись в одни и те же области памяти, поэтому требуется явная синхронизация со стороны программиста



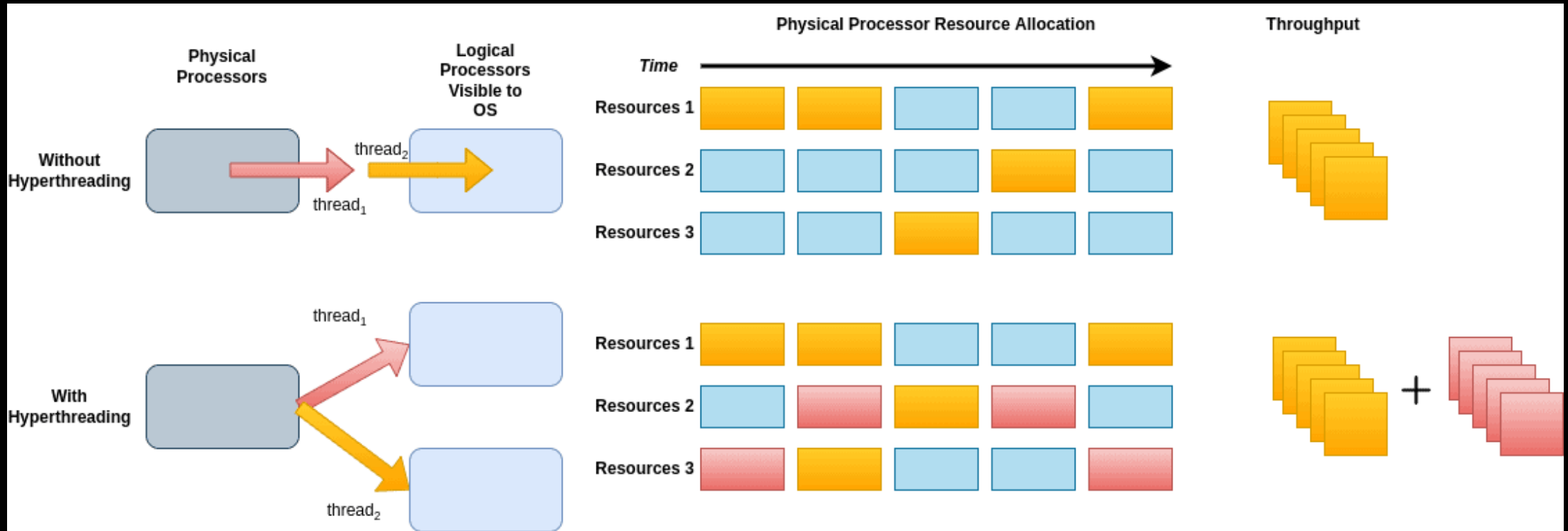
CPU



Все программы выполняются на процессоре. В современных реалиях везде используются многоядерные процессоры, т.е. процессоры содержащие 2 и более ядра на одном камне или в одном корпусе. Каждое из таких ядер может выполнять одновременно с другими выполнять задачи.

*Открыть lscpu или диспетчер задач, чтобы узнать инфу о процессоре

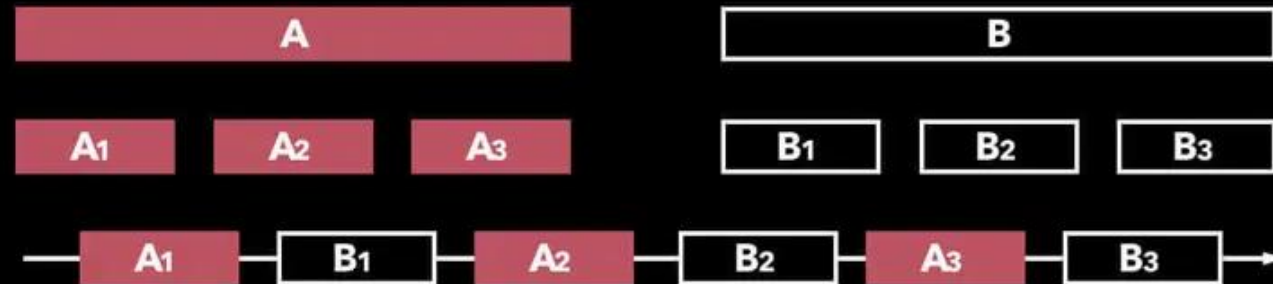
Hyper-Threading



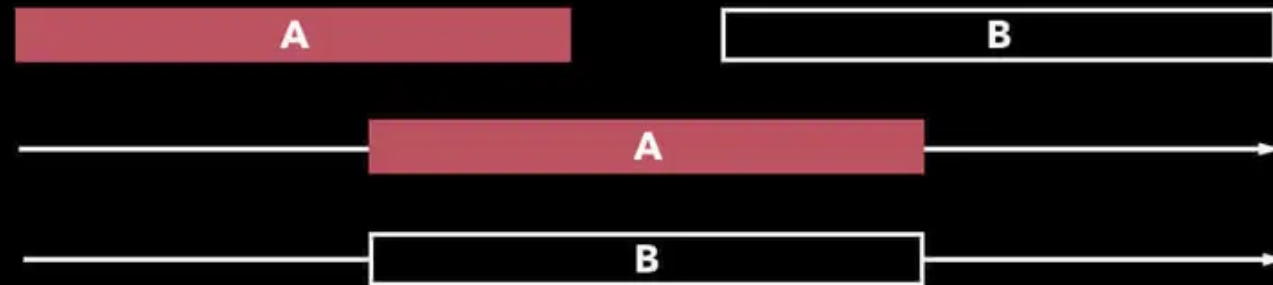
Технологию Гиперпоточности (Hyper-Threading или HT, у AMD процов аналог - SMT), если вкратце, то работает следующим образом: ядро хранит в себе состояние сразу двух потоков (регистры и т.д.) и для ОС это выглядит как два логических ядра. И если одно из логических ядер по той или иной причине находится в ожидании, второе логическое начинает работать. Также инструкции с обоих логических ядер могут перемешиваться и выполняться одновременно.

Конкурентность и параллелизм

Конкурентность - это выполнение задач за определённое время



Параллелизм - это одновременное выполнение нескольких действий



pthread

Для написания многопоточных программ мы с вами познакомимся с POSIX API, где POSIX (Portable Operating System Interface) — это семейство стандартов, определяющих общий набор API для Unix-подобных операционных систем

#include <pthread.h>

pthread_t — идентификатор потока;

pthread_create(); — Создать поток. Порядок запуска потоков из pthread_create случайный - определяет диспетчер задач (планировщик). Функция не блокирующая

pthread_join(); — Объединить две нити в одну (ждет завершения потока). Блокирующая функция

Не забываем линковать с библиотекой pthread: -lpthread

*Мануал с потоками — man pthreads

Race Condition

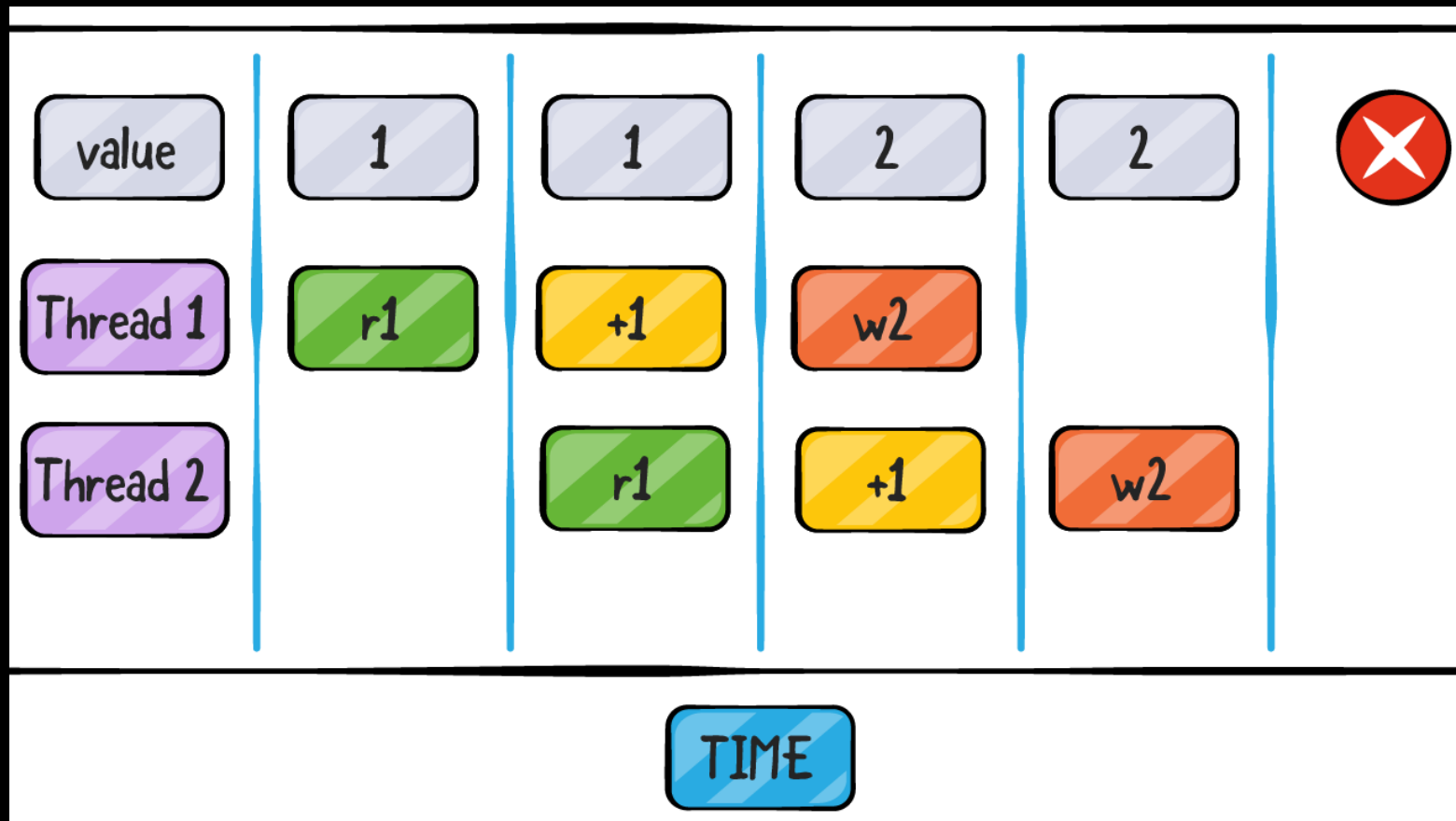
Есть разделяемая переменная `value = 1` и мы к ней одновременно хотим прибавить 1 в двух потоках и получить число 3

1. thread1 прочитал значение `value` => 1

2. поток1 прибавил 1 к значению `value` => 2;
поток2 прочитал значение `value` => 1

3. поток1 записал результат `value` => 2 в память; поток2 прибавил 1 к значению `value` => 2;

4. поток2 записал значение `value` => 2 в память.



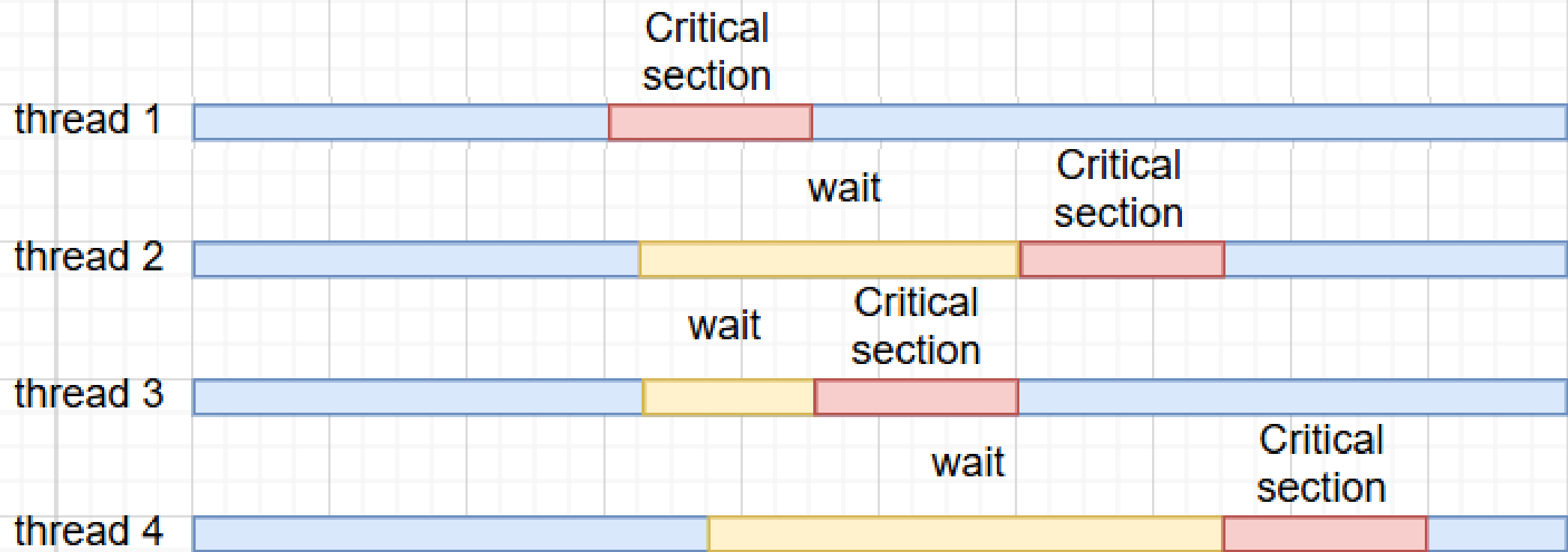
Примитивы синхронизации

Критическая секция — участок исполняемого кода программы, в котором производится доступ к общему ресурсу, который не должен быть одновременно использован более чем одним потоком выполнения. При нахождении в критической секции двух потоков возникает состояние «гонки» (race condition).

Mutex – Mutual Exclusion (Взаимное исключение), один из примитивов синхронизации, что помогает защитить критическую секцию с разделяемыми переменными.

```
pthread_mutex_t mutex; // mutex
pthread_mutex_init(&mutex, NULL); // или mutex = PTHREAD_MUTEX_INITIALIZER;
```

```
void *thread_func(void *arg) {
    ...
    pthread_mutex_lock(&mutex); // Блокируем mutex
    // critical section
    pthread_mutex_unlock(&mutex); // Разблокируем mutex
    ...
}
```



- 1) Поток1 попадает в lock()
- 2) Если lock() ни кем не занят, поток его занимает
- 3) Поток1 выполняет код критической секции. Все потоки, что попали на lock() ожидают
- 4) Поток1 завершил крит. секцию - разблокирует mutex. Следующий поток забирает lock()

Примитивы синхронизации

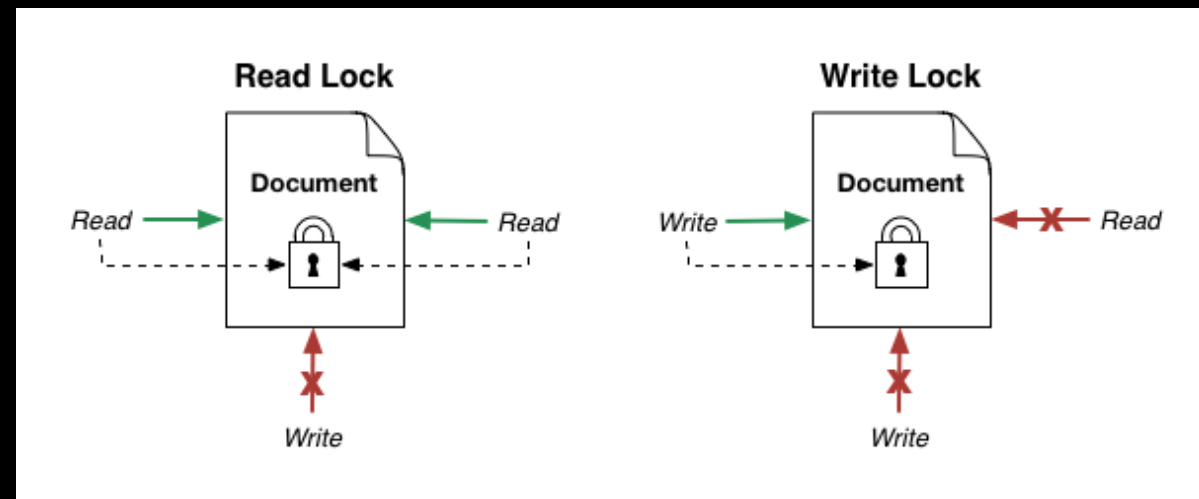
Операция в общей области памяти называется **атомарной**, если она завершается в один шаг относительно других потоков, имеющих доступ к этой памяти. То есть если несколько потоков одновременно обратятся к такой атомарной операции, то гарантированно выполнится в один момент времени лишь одна из них.

Семафор — примитив синхронизации работы процессов и потоков, в основе которого лежит счётчик над которым можно производить две атомарные операции: увеличение и уменьшение значения на единицу, при этом операция уменьшения для нулевого значения счётчика является блокирующей.

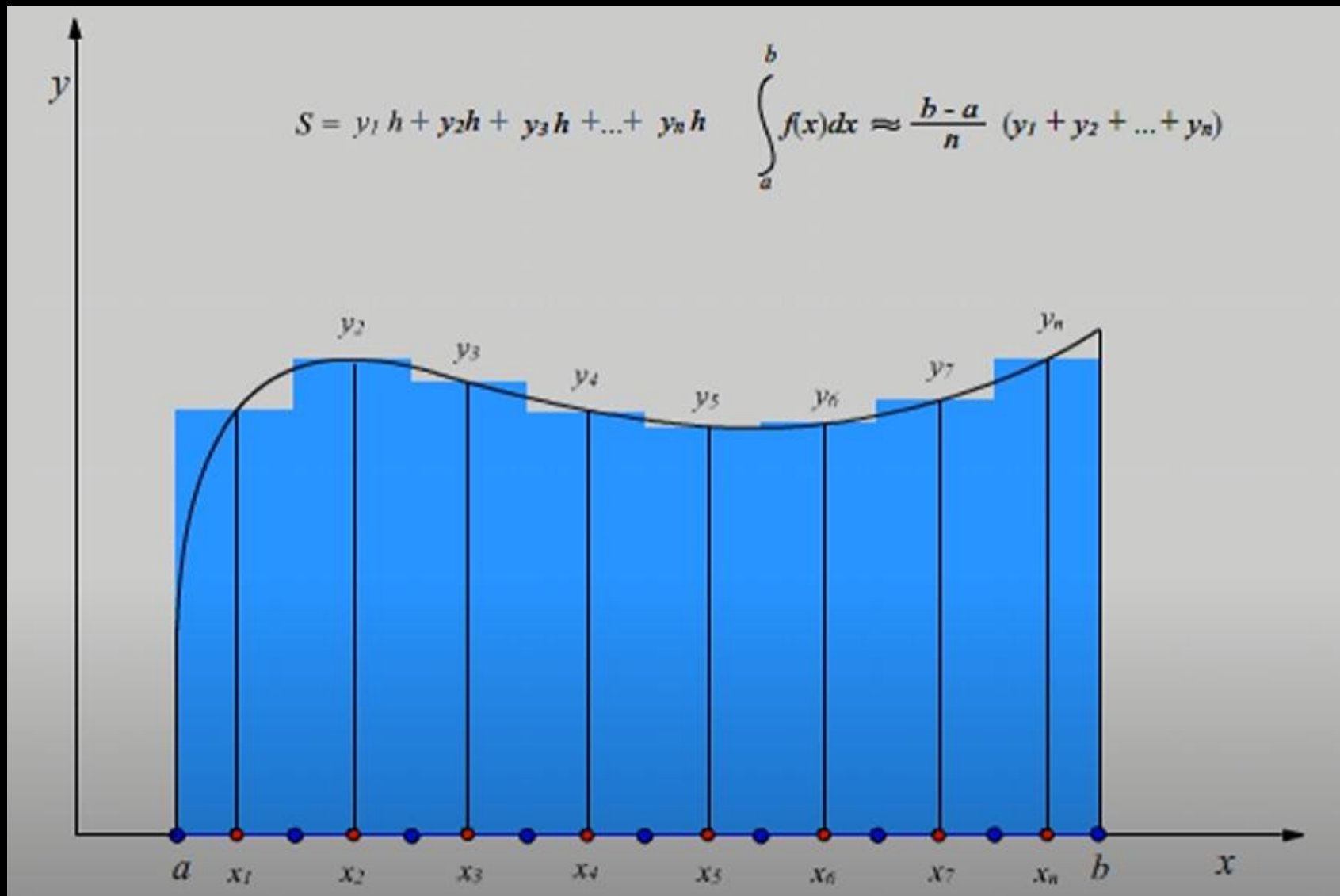
```
int sem_wait(sem_t *sem); // уменьшает semaphore (блокирует)  
int sem_post(sem_t *sem); // увеличивает semaphore (разблокирует)
```

Прочие блокировки:

spin lock
rw lock
rcu lock



Метод прямоугольников



Численное интегрирование

Гауссов интеграл

$$\int_a^b e^{-x^2} dx = \sqrt{\pi}$$

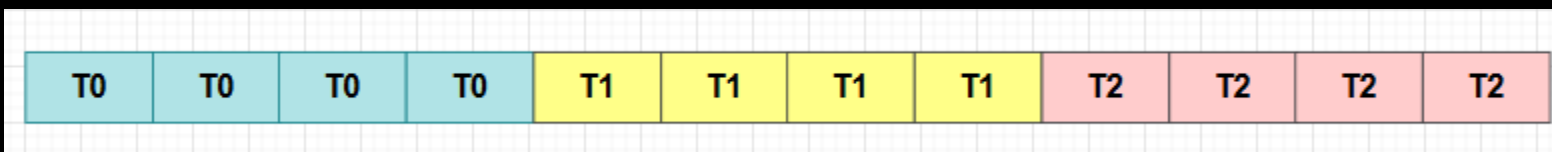
```
Start integration for 100000000 points with 1 thread [-5.000000, 5.000000]
my   pi = 3.141592653091
math pi = 3.141592653590

real    0m0.563s
user    0m0.554s
sys      0m0.000s
```

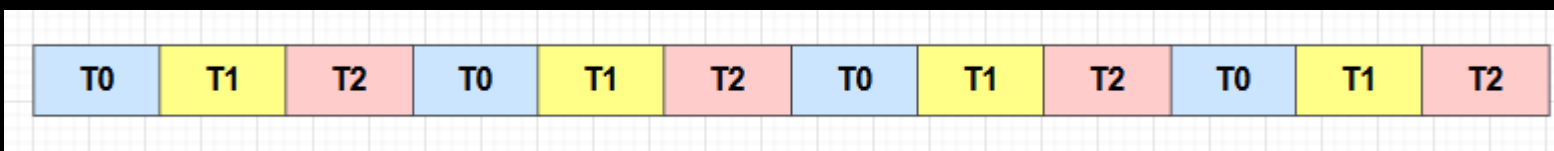
- Real is wall clock time - time from start to finish of the call. This is all elapsed time including time slices used by other processes and time the process spends blocked (for example if it is waiting for I/O to complete).
- User is the amount of CPU time spent in user-mode code (outside the kernel) within the process. This is only actual CPU time used in executing the process.
- Sys is the amount of CPU time spent in the kernel within the process.

Варианты распараллелить вычисления

- Разбить N элементов на обработку на количество обрабатываемых потоков таким образом, что $N/\text{число потоков}$ элементов обрабатывает первый поток, второй берет следующие $N/\text{число потоков}$ и т.д.



- Распределяем элементы на потоки циклически, т.е. сначала поток1, потом поток2, поток3 и т.д.



```
Start integration for 100000000 points with 2 threads [-5.000000, 5.000000]
thread 0 start for [0, 50000000)
thread 1 start for [50000000, 100000000)
my    pi = 3.141592653342
math  pi = 3.141592653590

real    0m0.310s
user    0m0.603s
sys     0m0.000s
```

Атрибуты потоков и мьютексов

`man pthread_attr_init`

- * Можно выбрать размер стека потока,
- * Его приоритет выполнения (scheduling priority)
- * Его алгоритм планирования (Scheduling policy) - тут есть разные планировщики (SCHED_FIFO, SCHED_RR, SCHED_OTHER)
- * Можно его отделить от родительского потока и тогда родительскому потоку не нужно ожидать завершения этого потока с `pthread_join` (такое ожидание будет вечным, т.к. дочерний поток не вернет результат родительскому). Однако при завершении родительского процесса поток с таким атрибутом все равно будет завершен.

`man pthread_mutexattr_init`

Чаще всего встречается такой атрибут как тип мьютекса (deadlocking, deadlock-detecting, recursive, etc)
Рекурсивный мьютекс позволяет захватывать один и тот же ресурс много раз без дедлока.

Deadlock (дедлок) - это зависание потока намертво при попытке залочиться дважды на одном и том же мьютексе

Условные переменные

global:

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;  
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
```

thread1:

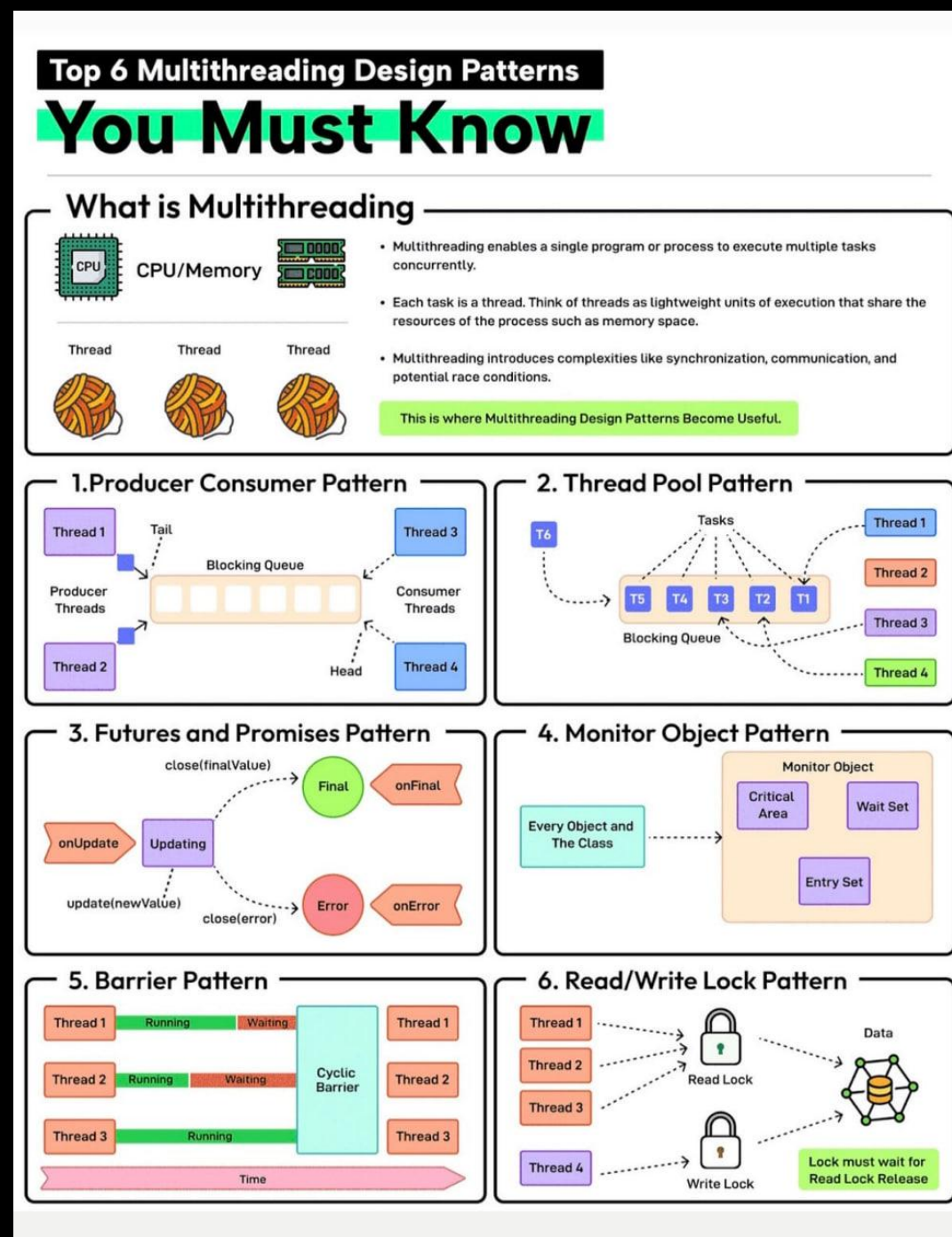
```
pthread_mutex_lock(&mutex);  
while (!condition)  
    pthread_cond_wait(&cond, &mutex);  
// do something that requires holding the mutex and condition is true  
pthread_mutex_unlock(&mutex);
```

thread2:

```
pthread_mutex_lock(&mutex);  
// do something that might make condition true  
pthread_cond_signal(&cond); // или pthread_cond_broadcast()  
pthread_mutex_unlock(&mutex);
```

Signal – будит лишь один поток, broadcast сразу всех

- Producer Consumer паттерн – классический пример с очередью. Например есть клиенты (производители), каждый из них в случайное время производит запрос к серверам (потребителям). Очередь к серверам одна. Повар (producer) готовит – Курьер (consumer) забирает.
- ThreadPool – набор потоков (пул), которые готовы решать задачи, поступающие из очереди. Например к нам поступают матрицы и нам необходимо посчитать от них обратную матрицу. Задача кладется в очередь тасков и потоки свободные эти задачки разбирают
- Futures and Promises паттерн относится к асинхронному программированию. Пример: заказываем еду в ресторане и получаем чек (Future) – способ обменяться на еду. Пока еда готовится, мы делаем что-то ещё (в телефоне сидим). Когда блюдо будет готово (Promise – обещание выполнено), ты обменяешь чек на еду.
"promise" - это то куда ИСПОЛНИТЕЛЬ положит результат
"future" - это то откуда ПОТРЕБИТЕЛЬ заберет результат
- Barrier паттерн – останавливает все потоки на определенной фазе выполнения и дает им работать дальше когда все завершат работу. Пример: Игра начнется когда все игроки загрузятся
- Monitor Object – Единственный объект в системе, который синхронизирует получение доступов
- RW lock – читатели читают всегда пока писатель не пишет, а писатель может писать лишь когда читатели не читают



Пример FIFO производитель-потребитель

<https://github.com/kruffka/C-Programming/blob/master/2024-2025/threads/src/fifo.c>

Пример threadpool

<https://github.com/kruffka/C-Programming/blob/master/2024-2025/threads/src/tpool.c>

Отладка многопоточных программ

Инструменты:

- gdb
- ThreadSanitizer

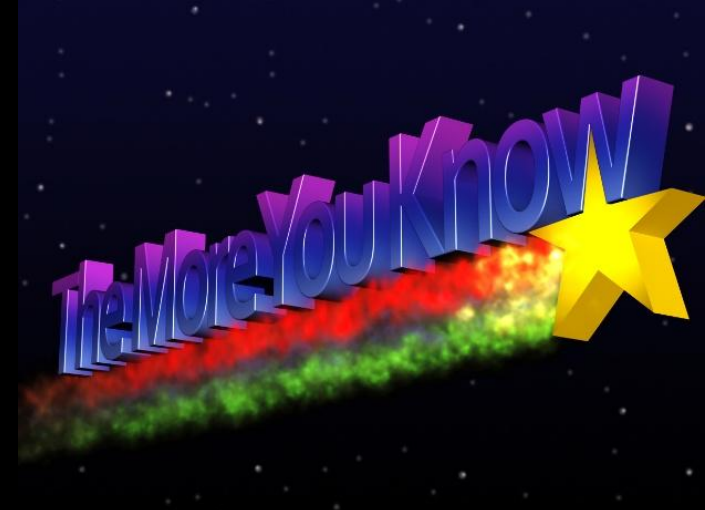
Info threads - Вывести информацию обо всех потоках
thread <thread> - переключиться на поток

```
(gdb) i th
  Id   Target Id                               Frame
* 1    Thread 0x7ffff7d8a740 (LWP 188606) "a.out" main (argc=
  2    Thread 0x7ffff7d89640 (LWP 188610) "a.out" __futex_abs
expected=0, futex_word=0x5555555581d0 <tpool_g+400>) at .
  3    Thread 0x7ffff7588640 (LWP 188611) "a.out" __futex_abs
expected=0, futex_word=0x5555555581d0 <tpool_g+400>) at .
  4    Thread 0x7ffff6d87640 (LWP 188612) "a.out" __futex_abs
expected=0, futex_word=0x5555555581d0 <tpool_g+400>) at .
  5    Thread 0x7ffff6586640 (LWP 188613) "a.out" __futex_abs
expected=0, futex_word=0x5555555581d0 <tpool_g+400>) at .
(gdb) □
```

CALL STACK

```
✓ a.out [196... PAUSED ON BREAKPOINT
    main(int argc, char ** argv)
✓ a.out [196999] PAUSED
    Show 5 More Stack Frames
    dequeue_task(task_t * task)
    worker(void * argP) tpo...
    Show 2 More Stack Frames
✓ a.out [197000] PAUSED
    Show 5 More Stack Frames
    dequeue_task(task_t * task)
    worker(void * argP) tpool.c
    Show 2 More Stack Frames
> a.out [197001] PAUSED
> a.out [197002] PAUSED
```

На заметку



- Атомарные операции

```
_Atomic uint32_t atomic_counter = 0;  
//...  
atomic_counter++; // все ок, атомарно будет  
atomic_counter = atomic_counter + 1; // будет гонка потоков, атомарность не сработает
```

- CPU Affinity

Идем в htop..

- SIMD - векторные инструкции (AVX, SSE и их друзья)

Гуглим SIMD mode vs Scalar mode

- lock-free алгоритмы

Алгоритмы работы с потоками, которые разделяют общую память, но не блокируют друг друга ради получения ресурса и имеют не попадают в состояние гонки

Литература

1. Blaise Barney. POSIX Threads Programming // <https://computing.llnl.gov/tutorials/pthreads>
2. Maurice Herlihy, Nir Shavit. The Art of Multiprocessor Programming. – Morgan Kaufmann, 2008. – 528 p
3. Эндрюс Г.Р. Основы многопоточного, параллельного и распределенного программирования. – М.: Вильямс, 2003. – 512 с.
4. «Экстремальный Си. Параллелизм, ООП и продвинутые возможности»
5. Darryl Gove. Multicore Application Programming: for Windows, Linux, and Oracle Solaris. – Addison-Wesley, – 480 p.
6. Спросить вашего любимого китайского кореша объяснить. А также на видеохостингах очень много неплохих объяснений



<https://github.com/kruffka/C-Programming>